

Fiche procédure Migration Symfony 5.4.45 → 7.3

Contexte :

Symfony 5.4.45 est la dernière version LTS de la branche 5.x, mais elle est désormais hors support actif. Pour des raisons de sécurité, de compatibilité avec les outils modernes et d'évolutivité, il est impératif de migrer vers la dernière version stable : Symfony 7.2.5 ou 7.2.6.

Symfony étant un framework structuré par versions majeures strictes, la migration ne peut se faire directement depuis 5.4 vers 7.2. Elle doit impérativement passer par plusieurs étapes intermédiaires :

- Symfony 6.0 : première étape pour entrer dans la branche 6.x, avec suppression des éléments dépréciés.
- Symfony 6.4 : nouvelle version LTS, étape de stabilisation.
- Symfony 7.0 : migration vers la branche 7.x (version non-LTS).
- Symfony 7.2.5 : version la plus récente, avec les dernières optimisations et correctifs.

Objectifs :

- Assurer une montée de version progressive et stable d'un projet Symfony jusqu'à la 7.2.5.
- Identifier les risques de compatibilité entre chaque version.
- Préserver l'intégrité fonctionnelle du projet tout au long des étapes.

Prérequis :

- PHP >= 8.1 Pour Symfony 6.
- PHP >= 8.2 Pour Symfony 7.
- Sauvegarde complète du projet (code, base de données, fichiers composer.json, config). Utilisation de Git afin de faire du versioning (Commit toutes les 3-5 dépréciations résolues).
- Environnement de test/d'intégration disponible.
- Connaissance minimale des composants Symfony / Utilisation de la doc au maximum.
- Recettage obligatoire après chaque passage d'une version à une autre.

Étapes de la migration :

Étape 1 : Migration vers Symfony 6.0

But : s'extraire de Symfony 5.x et basculer sur la nouvelle base technique (PHP 8+, typage strict, interfaces modifiées).

- Vérifier que toutes les dépréciations de Symfony 5.4 ont été corrigées.
- Adapter le code aux nouvelles interfaces :
 - getUsername() → getUserIdentifier()
- Mettre à jour les bundles tiers compatibles avec Symfony 6.
- Corriger les éventuels conflits dans composer.json.
- S'assurer que l'application fonctionne sans erreur en Symfony 6.0 (Recettage).

Étape 2 : Migration vers Symfony 6.4 (LTS)

But : Stabiliser le projet sur la dernière version LTS de la branche Symfony 6. Cette version permet de corriger les dépréciations restantes et de préparer la compatibilité avec Symfony 7.

- Modifier le fichier `composer.json` pour indiquer la version 6.4 de Symfony.
- Mettre à jour tous les composants Symfony et bibliothèques tierces compatibles avec Symfony 6.4.
- Corriger toutes les nouvelles dépréciations signalées par Symfony 6.4 (méthodes, paramètres, signatures...).
- Adapter les services, les contrôleurs, et les fichiers de configuration si des ajustements sont nécessaires (injection de dépendances, types, etc.).
- Vérifier la compatibilité des bundles utilisés avec Symfony 6.4.
- Tester toutes les fonctionnalités de l'application (formulaires, sécurité, API, tâches cron, etc.).
- Effectuer une recette fonctionnelle complète avant de passer à la version suivante.

Étape 3 : Migration vers Symfony 7.0

Passer dans la branche Symfony 7, qui supprime définitivement les éléments dépréciés dans les versions précédentes.

- Modifier le fichier `composer.json` pour cibler Symfony 7.0.
- Mettre à jour les dépendances du projet pour qu'elles soient toutes compatibles avec Symfony 7.
- Supprimer ou réécrire les parties du code qui utilisaient des fonctionnalités supprimées :
 - Vérifier les signatures des méthodes des interfaces (comme `UserInterface`).
 - Mettre à jour les événements et leurs noms (certains ont été renommés ou supprimés).

- Corriger les options de configuration supprimées ou modifiées (dans `security.yaml`, `framework.yaml`, etc.).
- Réviser les services customs et les extensions Twig si elles utilisent des fonctionnalités internes.
- S'assurer que tous les bundles tiers utilisés ont publié une version compatible avec Symfony 7.
- Lancer une phase de vérification fonctionnelle complète de l'application.

Étape 4 : Migration vers Symfony 7.2.6

But : Finaliser la migration en passant à la version stable la plus récente et la plus performante de Symfony.

- Mettre à jour la version de Symfony dans `composer.json` pour indiquer la version 7.2.6.
- Mettre à jour les composants Symfony et bibliothèques tierces compatibles avec cette version.
- Consulter le changelog officiel de Symfony 7.2 pour identifier d'éventuelles modifications ou nouvelles optimisations à intégrer dans le projet.
- Vérifier que l'application est entièrement compatible avec cette dernière version (aucune nouvelle dépréciation ou erreur).
- Si applicable, exploiter les nouveautés apportées par Symfony 7.2 (nouvelles annotations, composants améliorés, meilleures performances...).
- Et recetter l'application afin de voir si tout fonctionne parfaitement.

Étape 5 : Migration vers Symfony 7.3

But : Passer à la version la plus récente (7.3), publiée début juin 2025, afin de bénéficier des derniers correctifs, améliorations de performances et mises à jour structurelles avant la future LTS Symfony 8.0.

Pour pouvoir migrer vers la version 7.3 de Symfony, il faut :

- Mettre à jour les dépendances Symfony vers la version `^7.3`.
- Mettre à jour les bundles tiers compatibles (notamment ceux liés à la sécurité et au serializer).

- Enfin, corriger les dépréciations apparus dans Symfony 7.3.

Exemple d'une dépréciation apparu à cause de la migration vers Symfony 7.3 :

VoteOnAttribute :

L'interface Voter attendra un quatrième argument qui est (?Vote \$vote) dans Symfony 8. Une dépréciation apparaît donc pour nous prévenir et préparer le code maintenant (si on le souhaite) pour la prochaine version majeure (8).